STICK ARENA

Project Report and Presentation Guide

Godot 4.7 local 2D fighting-game prototype

PROJECT SUMMARY

Playable local stick-fighting game with 1 Player, 2 Players, Practice Mode, CPU difficulty, weapons, super moves, round timer, two stages, menus, and test evidence.

HOW TO PRESENT IT

Use this report for the project story and features. Use the companion Frontend and Backend Architecture Guide for detailed system diagrams.

26 pages | prepared for academic project presentation

This report records the project details and the requested development history. It describes a local game; it does not claim online multiplayer or server infrastr

2. Project map and report purpose

A presentation-ready reference for the current Stick Arena prototype.

WHAT THIS REPORT IS

A single project document for explaining the game to a teacher. It combines the development brief, implemented systems, controls, architecture, testing evidence, and delivery status.

HOW TO USE IT

Read Sections 1-14 for the product and technical overview. Use Sections 15-18 to explain testing, GitHub delivery, and a short demonstration. The final pages record the requested feature history.

COMPANION DOCUMENT

The separate **Frontend and Backend Architecture Guide** contains the detailed 60-page visual architecture material. This report deliberately summarizes it instead of duplicating every diagram.

IMPORTANT TERMINOLOGY

The project is a local Godot game. In this report, **backend** means local gameplay logic inside Godot - it is not an online server, database, or multiplayer service.

PRESENTER TAKEAWAY

Start your presentation by saying: *Stick Arena is a locally playable 2D fighting-game prototype built in Godot 4.7. The focus was a complete and testable gameplay loop, not commercial-scale online infrastructure.*

3. Executive summary

What was built and why it is suitable for a student game-development presentation.

PROJECT GOAL

Create a small playable fighting prototype with two stick fighters, local movement, combat, health, rounds, menus, practice tools, a CPU opponent, two stages, and a browser-delivery path.

CURRENT RESULT

The game supports 1 Player versus CPU, 2 Player local play, and Practice Mode. A normal match is best-of-three, uses 60-second rounds, supports weapons and special moves, and includes a lava-hazard stage.

TECHNOLOGY

Godot 4.7 with the GL Compatibility renderer. The game uses GDScript, scene files, built-in shapes, particles, shaders/tweens, labels, bars, and audio nodes. No external character art is required.

SCOPE DISCIPLINE

The work intentionally avoids online multiplayer, progression, accounts, mobile touch controls, complex external assets, and advanced networking. Those would be future extensions, not requirements for the prototype.

PRESENTER TAKEAWAY

The strongest proof of success is that the project progressed from one controllable fighter to a complete local match loop with menus, modes, CPU behavior, stage hazards, testing, and public source control.

4. Original playable-prototype milestone

The first requested implementation target and the way it shaped the project.

INITIAL MILESTONE

Create a simple 2D arena and one controllable stickman using placeholder shapes. Implement A/D movement, jumping, gravity, and floor collision while keeping the structure beginner-friendly.

MONDAY PROTOTYPE TARGET

Show two visible fighters; player movement and jumping; ground collision; attacks; damage and knockback; health; winner screen; and restart. This kept the early build achievable in short daily work sessions.

INCREMENTAL DELIVERY

Work was requested in small layers: base movement, combat, HUD, rounds, weapons, effects, CPU, practice, menus, stage selection, test documentation, repository publishing, and browser export.

PLACEHOLDER-FIRST DESIGN

Stick figures and arena visuals are made from Godot nodes and simple geometry. This makes gameplay testable before investing time in external sprites, animation tools, or custom art production.

PRESENTER TAKEAWAY

Explain that the project was planned as an iterative prototype. Each layer was tested before the next was added, which is a practical approach for a short school deadline.

5. Modes and match structure

The three ways a user can play Stick Arena.

1 PLAYER

The human uses Player 1 controls. Player 2 is controlled by the built-in CPU, which reacts after a difficulty-based delay and uses the same combat rules as a human fighter.

2 PLAYERS

Two people share one keyboard. Player 1 uses A/D, W or Space, F, S, and G. Player 2 uses arrow keys, O, Down, and P. This is the primary local versus mode.

PRACTICE

A training mode with no normal timer or match win condition. Fighters can be tested against a CPU or a passive dummy. Defeated fighters recover instead of ending the round.

BEST-OF-THREE

Normal matches are first to two round wins. A match ends as soon as one fighter reaches two victories. The match-end screen offers rematch support through the existing restart key.

PRESENTER TAKEAWAY

Emphasize that one rules foundation is shared by all modes. Practice changes only match flow; it does not create a separate combat system.

6. Controls and player input

Keyboard controls presented on the start screen, controls screen, and in the game README.

PLAYER 1

A / D move left/right. **W or Space** jump. **F** attack. **S** block. **G** special move when the super meter is full.

PLAYER 2

Left / Right move. **Up** jump. **O** attack. **Down** block. **P** special move when the super meter is full.

MENUS AND MATCH

Up / Down navigate menus. **Enter** confirms. **Left / Right** changes selected settings or stage. **Esc** returns or opens pause. **R** restarts/rematches.

PRACTICE-ONLY TEST INPUT

M fills Player 1's meter in Practice Mode. The implementation also supports a Player 2 meter-fill key in practice, enabling special-move testing without waiting for repeated hits.

PRESENTER TAKEAWAY

When demonstrating, open Controls from the main menu first. It proves the player can understand the keyboard mapping before a match starts.

7. Fighter movement and collision

The core physics behavior that makes the game playable.

MOVEMENT

Each fighter is a CharacterBody2D. Horizontal input accelerates toward a movement speed. Gravity applies while airborne, and a jump velocity launches the fighter from the floor.

ARENA SAFETY

Arena 1 keeps fighters within the normal arena bounds. Fighters cannot simply walk through one another; the main controller keeps spacing valid while preserving close-range combat.

FACING

Fighters automatically face their opponent. This maintains correct attack direction and means players do not need a separate turn-around key.

FEEDBACK POSES

Idle bob, walk swing, attack, block, recoil/stagger, and collapse poses are implemented through simple visual transforms and eased tween motion rather than external animation files.

PRESENTER TAKEAWAY

The movement system deliberately stays simple: it is easy to explain, easy to test, and gives the combat system predictable positioning.

8. Unarmed combat, blocking, and impact feedback

The main melee interaction and its fixed gameplay values.

NORMAL PUNCH

A normal attack deals **20 damage**. It uses a short attack range, checks the fighter's facing direction, applies the existing knockback of **(380, -180)**, and shares the normal cooldown.

COOLDOWN

After throwing a punch or using a weapon swing, a fighter waits about **0.45 seconds** before another attack. This prevents a held attack key from producing unlimited hits.

BLOCKING

Player 1 holds S and Player 2 holds Down. A blocked normal attack deals **5 chip damage** and causes no knockback. Blocking therefore reduces damage but does not make a fighter completely invulnerable.

HIT FEEL

On a landed hit, the target briefly flashes white, recoils, the camera shakes if enabled, and impact effects/sound are played. A non-blocked hit uses a stagger pose for clearer feedback.

PRESENTER TAKEAWAY

State the values exactly if asked: punch 20, blocked chip 5, normal knockback (380, -180), and a 0.45-second cooldown.

9. Weapon pickup and armed attack

A simple bat/stick system layered on top of the existing punch rules.

SPAWN AND PICKUP

At the start of a round, one rectangle-shaped bat spawns at a random floor position. A fighter collects it by walking over it; it disappears from the ground and visually appears in the holder's hand.

ARMED SWING

The attack button performs an armed swing while holding the bat. It keeps the existing cooldown but has a larger range and deals **35 damage** instead of the 20-damage unarmed punch.

DISARM

Any hit on an armed fighter, blocked or unblocked, drops the weapon at the fighter's current position. Either fighter can then collect it, creating a short contest for map control.

HUD STATE

A weapon indicator appears near the health information of the fighter holding the bat. The player can understand weapon ownership without watching every pickup closely.

PRESENTER TAKEAWAY

The weapon changes one attack only. It does not replace the normal movement, meter, round, or block systems, which keeps the feature easy to balance and explain.

10. Super meter and energy blast

The high-impact system added after normal combat was stable.

METER RULES

The meter starts at 0 at the beginning of each round. It caps at 100 and fills when fighters trade successful hits. The HUD renders it beneath each health bar and pulses when full.

ACTIVATION

At a full meter, Player 1 presses G and Player 2 presses P. The fighter uses a short charge pose and launches a fast horizontal energy projectile. Only one blast per fighter is allowed on screen.

DAMAGE AND BLOCK

A special blast deals **40 damage** with strong knockback. If blocked, it deals **10 chip damage**. The meter resets after use and at the beginning of a new round.

EFFECTS

The blast uses a glowing projectile appearance, trail particles, firing sound, hit sound, and an impact effect. It disappears on a fighter hit or when leaving the arena.

PRESENTER TAKEAWAY

Describe the super move as a reward for successful fighting, not a random power. The meter introduces a simple risk-and-reward decision without altering the base punch rules.

11. Rounds, timer, and winner logic

How a normal match moves from FIGHT to the result screen.

ROUND OPENING

The start flow announces Round 1, Round 2, or Round 3, then briefly shows FIGHT!. Round-win indicators keep track of each fighter's progress toward two round victories.

CLOCK

Each normal round is 60 seconds. The large countdown appears between the health bars. It starts with FIGHT! and stops during pause, KO, and the match-end screen.

TIME-OUT

At 0 seconds, TIME! appears. The fighter with more health wins. If both health values are equal, it is a draw: the round is replayed and neither player gains a round win.

KO AND MATCH

A fighter at zero health loses the round. The losing stick figure collapses. First to two round wins gets the clear PLAYER X WINS THE MATCH screen and can rematch with R.

PRESENTER TAKEAWAY

This is a complete match loop: intro, fight, timer or KO decision, round score, next round, and match conclusion. It is the best sequence to demonstrate to a teacher.

12. Single-player CPU and difficulty

A rules-based local opponent, not an external AI service.

CORE BEHAVIOR

The CPU walks toward Player 1 until it reaches attack range, attacks after a small random delay, blocks or backs away when threatened, and jumps occasionally.

FAIRNESS

The CPU calls the same fighter actions and uses the same cooldown, damage, weapon rules, block rules, meter, and round rules as a human. It has an intentional reaction delay of roughly 0.2 to 0.3 seconds.

DIFFICULTY

Easy, Normal, and Hard change only decision behavior: reaction delay, block chance, and attack frequency. The game values for punch, bat, special, knockback, and match rules do not change.

AWARENESS

The CPU looks for nearby floor weapons, avoids deliberately stepping into lava, and fires its full-meter special from mid-to-long range. It does not wait forever just because the player might block or dodge.

PRESENTER TAKEAWAY

If asked about AI, say it is transparent gameplay logic using distance, timing, and difficulty parameters. That is appropriate for a fair and explainable student prototype.

13. Arena 1 and Arena 2

Two selectable stages with shared combat rules and distinct mechanics.

ARENA 1

The standard stage uses the full-width solid floor and normal arena bounds. It is the safest stage for learning the controls and demonstrating the core combat loop.

ARENA 2 LAYOUT

The second stage has a narrow central safe platform with lava pits on both sides. It uses a separate sky/skyline color and a visually distinct floor/lava treatment.

LAVA HAZARD

A fighter standing or falling in lava takes **10 damage every 0.5 seconds**. A brief upward/inward knockback helps the fighter escape instead of becoming permanently stuck.

WEAPON HAZARD

A dropped bat entering lava disappears. The weapon system schedules a new bat to spawn back on the safe platform after a short delay, preserving the pickup system.

PRESENTER TAKEAWAY

Arena 2 changes position strategy but not the basic damage values. A player can knock an opponent toward danger, while the CPU is designed not to walk into it intentionally.

14. Practice Mode and debugging support

A training space that lets the player test mechanics safely.

NO NORMAL MATCH END

Practice disables the standard 60-second timer and best-of-three conclusion. If health would normally reach zero, the fighter recovers to full after a short delay instead of ending the round.

OPPONENT CHOICE

Practice can use the normal live CPU, with the same difficulty selection, or a passive dummy. The dummy does not move or attack but can still be hit, blocked against, and knocked around.

TESTING FEEDBACK

The exact damage number briefly appears above a fighter when they are hit. This makes it easy to verify 20 punch damage, 5 chip damage, 35 bat damage, and special behavior.

METER TESTING

A dedicated practice input fills the meter. This makes the energy-blast system easy to demonstrate and test without first producing many successful attacks.

PRESENTER TAKEAWAY

Practice Mode is a useful teaching feature: it turns the game into a safe test environment and demonstrates that the project supports verification as well as entertainment.

15. Main menu, HUD, and pause screens

The player-facing presentation layer and navigation flow.

MAIN MENU

On launch, the player sees STICK ARENA and the options Play, Settings, and Controls. Play leads to mode, difficulty, and stage selection. Esc does nothing at the top level.

SETTINGS

Settings include a master-volume slider and a screen-shake on/off option. Volume affects game sounds, while the toggle controls the existing landed-hit camera shake.

HUD

Rounded panels support health bars, super bars, names, timer, round labels, weapon indicators, and round-win indicators. Health changes from green to yellow to red as it decreases.

PAUSE

Esc pauses a current match. The pause screen keeps Resume and Restart and adds Main Menu. It is available in Practice Mode as well as normal fights.

PRESENTER TAKEAWAY

The HUD is not just decoration: it communicates health, super readiness, weapon state, score, timer status, and whether the opponent is CPU-controlled.

16. Visual polish, camera, particles, and sound

Built-in Godot polish without external art assets.

BACKGROUND AND FLOOR

The flat background was upgraded to a layered sky gradient, silhouette skyline/crowd look, and patterned floor. Arena 2 has a separate lava-oriented look.

FIGHTERS

Stick figures use thicker lines, simple shading, and a floor shadow. Their motion uses eased tweens so idle, walking, attack, block, recoil, and collapse poses do not look purely linear.

EFFECTS

Small sparks mark a hit, dust appears from movement/landing, and a larger burst appears on a KO. Lava includes a pulsing glow and rising ember particles.

CAMERA AND AUDIO

The camera frames both fighters, adds a short KO punch-in, and supports hit shake. Sound effects cover a landed hit, round/match end, special firing, and special impact.

PRESENTER TAKEAWAY

All presentation improvements are native Godot-node work. The prototype looks more finished while still remaining lightweight and explainable.

17. Godot scene and script structure

A small, maintainable layout used throughout the prototype.

MAIN SCENE

scenes/game/main.tscn is the assembly point. It contains the arena, both fighter instances, the weapon pickup, and a CanvasLayer HUD.

MATCH CONTROLLER

scripts/game/main.gd controls menus, matches, rounds, timer, CPU decisions, stage selection, weapon spawning, projectiles, HUD refreshes, effects, and audio triggers.

FIGHTER COMPONENT

scenes/player/fighter.tscn creates the reusable fighter body and visuals. **scripts/player/fighter.gd** owns per-fighter movement, attacks, health, block state, weapon state, and super-meter state.

ARENA COMPONENT

scenes/arenas/arena.tscn provides ground and walls. Main logic adjusts behavior for Arena 1 versus the Arena 2 lava platform/hazard rules.

PRESENTER TAKEAWAY

This division keeps the code beginner-friendly: each fighter handles itself, while main.gd coordinates things that involve the whole match.

18. Frontend and backend summary

The relationship between what players see and the local logic that drives it.

FRONTEND

Menus, selection screens, arena visuals, fighter drawings, animations, HUD, timer label, health/super bars, particles, screen tint, camera, and sounds are player-facing systems.

LOCAL BACKEND

Input interpretation, physics, cooldown checks, combat hits, damage, knockback, blocking, CPU intent, timer checks, round score, weapon respawn, and lava damage are local game rules.

CONNECTION

Fighter and match events update values. The controller then refreshes HUD bars, labels, effects, and match overlays. The detailed companion architecture PDF breaks these flows into individual diagrams.

NO SERVER

There is no remote database, login system, account service, or online networking. The browser version runs the same local logic on the player's device.

PRESENTER TAKEAWAY

Use the phrase *presentation layer and local game logic* if your teacher asks why the project has frontend and backend diagrams even though it is not a web-server application.

19. Testing approach and known evidence

What was checked, and what still requires manual play testing.

AUTOMATED SMOKE CHECKS

The recorded Godot 4.7.2 headless smoke run completed **40 checks with 0 failures**. The main scene was also launched for 90 frames without scene or script errors.

MANUAL CASES

The project has manual test coverage for menus, movement, jumping, attacks, blocking, rounds, timer, special, weapons, CPU, stages, Practice Mode, and pause/restart behavior.

WHAT AUTOMATION PROVES

Headless checks are useful for script errors, scene startup, and repeatable rule-level checks. They are fast and do not require a player at the keyboard.

WHAT NEEDS HUMAN REVIEW

A human should still verify physical keyboard feel, visual framing, animation quality, browser audio, sound balance, and performance on the device used for the presentation.

PRESENTER TAKEAWAY

Be accurate: say the test run proves the checked logic started correctly, but visual and input feel still deserve manual verification. This makes your presentation credible.

20. GitHub, documents, and browser plan

How the project is packaged for review and sharing.

PUBLIC REPOSITORY

The source project is stored in a public GitHub repository. It contains the Godot project, scenes, scripts, export configuration, README, and selected presentation documents.

WEB EXPORT

A Web export preset and GitHub Actions workflow were prepared to build the Godot project for browsers. Godot's GL Compatibility renderer is the correct renderer path for WebGL-based browser export.

WEBSITE REALITY

A browser build lets a player run the game without installing Godot. The developer still uses Godot to create and update the game. Browser sound may need a click or key press before audio begins.

REPOSITORY DOCUMENTS

The intended final repository-document set is this project report plus the detailed Frontend and Backend Architecture Guide. Other test details are summarized in this report rather than kept as a separate document.

PRESENTER TAKEAWAY

Show the GitHub repository at the end of the presentation. It demonstrates version control, documentation, and a deliverable others can inspect.

21. Suggested 5-minute demonstration

A short flow that shows the strongest working parts without getting lost in menus.

MINUTE 1 - OVERVIEW

Open the GitHub repository and explain the project goal, Godot version, and local-fighting scope. Mention the three modes and two stages.

MINUTE 2 - MENU

Show the main menu, Settings, Controls, then choose 1 Player. Pick a difficulty and Arena 1. Explain that these screens were added for a complete user journey.

MINUTES 3-4 - MATCH

Demonstrate movement, jumping, one punch, blocking, health bars, round timer, weapon pickup, super-meter fill, and special move. If time allows, show a KO or time-out.

MINUTE 5 - SPECIAL FEATURES

Open Practice Mode or Arena 2. Show damage numbers, passive dummy/CPU selection, meter fill, lava hazard, and pause menu. Close by showing the two PDF documents in the repo.

PRESENTER TAKEAWAY

Do not try to demonstrate every feature live. A clean five-minute story is more impressive than a rushed tour of every menu.

22. Requested development history: foundation

A record of the original requirements supplied during the project conversation.

SETUP AND DEADLINE

The project began with Godot 4.7, VS Code, Git/GitHub, and Kilo Code configured. The requested target became a playable prototype for Monday, built in short daily sessions.

FIRST BUILD

Requested: a 2D arena, one controllable placeholder stickman, left/right movement, jumping, gravity, and floor collision. The instructions stressed minimal, beginner-friendly code and no advanced systems at first.

EARLY GAME LOOP

Requested prototype additions: a second player, local movement, jumps, attacks, damage, knockback, health, winner screen, restart, and simple UI. Weapons, AI, mobile controls, online play, and fancy assets were initially out of scope.

CORE EXPANSION

Requested next: A/D for Player 1 and arrows for Player 2; bounds and player spacing; auto-facing; best-of-three rounds; Round/FIGHT messages; round indicators; health bars; sounds; and a start screen with controls.

PRESENTER TAKEAWAY

This page shows that the game developed from a small playable core. The later systems were not imagined all at once; they were added as stable extensions.

23. Requested development history: combat and polish

Features requested on top of the basic fighting system.

DEFENCE AND FEEDBACK

Requested: attack cooldown without changing existing damage/knockback, blocking with chip damage/no knockback, white flash, hit shake, and a match-end screen. The requirement preserved the existing fighter.gd/main.gd/main.tscn structure.

WEAPON AND ANIMATION

Requested: random bat pickup, hand-held indicator, 35-damage longer-ranged armed swing, disarm on any hit, plus idle bob, walk swings, recoil, and a collapse pose.

VISUAL PASS

Requested: layered sky/skyline/floor, thicker shaded fighters and shadows, hit/dust/KO particles, rounded HUD panels, camera framing and KO zoom, vignette/color tint, and eased motion.

TIMER

Requested: 60-second round timer, start with FIGHT!, pause during appropriate states, health-based time-out win, draw replay, red pulse in the final 10 seconds, and TIME! before the round result.

PRESENTER TAKEAWAY

When asked why values are consistent, explain that later requests repeatedly required preserving existing damage, knockback, cooldown, controls, and round rules.

24. Requested development history: modes, AI, stages

The requested additions that made the prototype more complete.

CPU AND DIFFICULTY

Requested: 1 Player mode, CPU decisions, same combat rules and reaction delay as humans, weapon awareness, Easy/Normal/Hard difficulty, and CPU naming in the HUD/match-end screen.

SUPER MOVE

Requested: super meters, a full-meter projectile with 40 damage/10 chip, one blast per fighter, meter reset rules, visuals/sounds, CPU use, and new controls on start/pause screens. A later issue reported the meter not filling fully and it was addressed.

MENUS AND STAGES

Requested: Play/Settings/Controls menu, master volume, shake toggle, Controls back navigation, Main Menu from pause, then a stage-selection step with Arena 1 and Arena 2.

LAVA ARENA AND PRACTICE

Requested: Arena 2 safe platform/lava hazards, lava damage and weapon respawn, CPU avoidance; later a Practice mode with no standard timer/win condition, CPU/dummy choice, recovery, meter fill, damage numbers, and both arenas.

PRESENTER TAKEAWAY

The project is still a prototype, but these features make it feel like a small complete game rather than only a movement demonstration.

25. Requested development history: quality and delivery

Testing, documents, publishing, and presentational requirements.

TESTING

Requested: test-case documents, self-checking/runtime results, a review of partial/non-running behavior, and clear result reporting. The project therefore includes an automated smoke-test record and manual-test guidance.

DOCUMENTATION

Requested: frontend and backend diagrams, PDF delivery, a two-presenter split so a friend could explain part of the work, an in-depth architecture guide, and later a 50+ page diagram document.

REPOSITORY AND PRESENTATION

Requested: make the repository public, help with a LinkedIn post, check GitHub connection, and prepare the game for presentation to a teacher. The public repository is now the central deliverable.

WEBSITE

Requested: make the game playable as a website rather than requiring Godot. A Godot Web export preset, landing page, and GitHub Actions deployment workflow were prepared; public Pages activation was also configured.

PRESENTER TAKEAWAY

This report collects the requested history so you can explain why each major feature exists and how it connects to the original presentation deadline.

26. Conclusion and future work

A clear final message for the teacher presentation.

WHAT IS COMPLETE

A functioning local stick-fighting prototype: menus, 1P/2P/Practice, CPU, normal and weapon combat, block, super meter, rounds/timer, two stages, visuals, audio hooks, testing evidence, documentation, and public GitHub delivery.

WHAT IS INTENTIONALLY NOT INCLUDED

Online multiplayer, accounts, persistent progression, paid assets, mobile touch controls, multiple maps beyond the two prototypes, and commercial balancing are outside the planned school-project scope.

FUTURE IMPROVEMENTS

Possible next steps are browser-play testing across devices, final audio balancing, keyboard-feel polish, additional weapons/stages, improved CPU patterns, custom artwork, and later a separate online-multiplayer design.

FINAL STATEMENT

Stick Arena demonstrates a full small-game development loop: define a restricted goal, build incrementally, test the critical behaviors, document the architecture, and publish the source and presentation material.

PRESENTER TAKEAWAY

Finish with: *The project is designed as a working prototype. Its main achievement is not only the number of features, but that they run together inside one consistent local game loop.*